

充电桩运维管理 AI Agent 平台 —— 项目答辩稿

配套 PPT：充电桩运维管理AI-Agent平台-项目答辩.pptx（12 页，与本文档逐页对应）

建议时长：10 分钟讲解 + 10 分钟问答

语速参考：230 字/分钟。加粗处是重音，|| 表示停顿。

数据口径：本文档所有数字均为本机实测，依据写在 PPT 页脚与本文档第七节。

代码中 graph.py:202-203、nodes.py:13、agent_service.py:178、models/ai.py:34 仍残留「interrupt 挂起/恢复」的旧注释，**答辩时请按「两阶段执行 + 数据库持久化」表述**，否则会 与 nodes.py:1444-1454 的工程取舍说明自相矛盾。

一、时间轴（贴在手边）

页码	时间	段落	任务
01	0:00-0:20	开场	一句话说清这个平台干什么
02	0:20-0:50	五个数字	规模、表数、接口、Agent、验证
03	0:50-1:30	硬约束代码化	子任务公式，全场技术含量最高
04	1:30-2:15	AI 工作流	18 节点 + 两个工程难点
05	2:15-2:50	八类 Agent	分三组讲，不逐个念
06	2:50-3:20	数据权限	四范围 + 三个易错点
07	3:20-3:55	验证结果	182 项 + 联调真实验证到的链路
08	3:55-4:25	已知限制	主动交代，不等老师问
09	4:25-4:50	系统架构	四层 + 规则版本化
10	4:50-5:50	三个问题	全场重点 ，三个真实的坑
11	5:50-6:50	关键取舍	预设问答五条
12	6:50-7:10	总结	落点 + 致谢
—	7:10-13:00	现场演示	见第五节动线
—	13:00-20:00	问答	见第六节

如果只给 8 分钟：砍掉第 09 页，第 11 页只讲前三条。**第 03、07、10 页不能砍。**

二、逐字稿

01 开场 (0:00-0:20)

【01】 各位老师好。我汇报的是**充电桩运维管理 AI Agent 平台**。

【01】 一句话说清它是干什么的：它把巡检、工单、故障、报告这四条运维主线的规则全部代码化，让 8 类 AI Agent 在硬约束之下辅助调度与决策。

【01】 整个平台的设计主线只有一句：硬约束代码化，LLM 只负责解释与表达。||

02 五个数字 (0:20-0:50)

【02】 先用五个数字说清规模。

【02】 第一，代码量。后端 17502 行（51 个 Python 文件），前端 11890 行（28 个 Vue 页面 + 4 个 JS + 1 个 CSS），合计 29392 行。

【02】 第二，数据模型。32 张表，117 个索引——模型定义 32 张，实际跑的库也是 32 张，完全一致。

【02】 第三，接口。103 个接口路径、114 个操作。其中 AI Agent 29 个、管理后台 23 个、工单作业 14 个。

【02】 第四，Agent。8 类 AI Agent，由 18 个节点的 LangGraph 工作流编排。

【02】 第五，验证。182 项验证全部通过——后端冒烟 96 项、真实服务联调 54 项、前端模块编译 32 项。

【02】 平台覆盖四条业务主线：巡检（五类工单、子任务按公式计算）、工单（状态机约束、多方案比选、人工确认）、故障（草稿-上报-核查闭环、自动判定等级、多图上传）、报告（日/周/月/即时四类、六大内容模块）。

03 硬约束代码化 (0:50-1:30) ★ 技术含量最高

【03】 这一页我想重点讲：子任务数量按公式算出来，不靠人估。

【03】 系统里有五类工单，每一类的子任务数量都有明确公式：

【03】 巡视、设备检查、其他这三类，子任务数 = 站点数量 × 巡检周期 × 巡检频率；

【03】 特巡 = 站点数量 × 巡检次数；

【03】 消缺 = 固定 1 个子任务。

【03】 其中「巡检频率」是一张固定的映射表：日巡检 = 30、周巡检 = 4、月巡检 = 1。||

【03】 为什么这些必须写进代码，而不是交给模型判断？

【03】 因为子任务数直接决定派工量、工时估算和考核口径。算错一个，整月的报表全部失真。

【03】 实现上，公式只有一份——放在 `services/work_order.py` 的 `calc_subtask_count()`，求解器和接口共用同一份实现，不重复写第二遍。

【03】 而且求解器在生成方案之前会复核：巡视类必须给出巡检周期与频率；特巡次数至少为 1；消缺只能对应单个站点。不满足直接判为不可行。

【03】 我给的判断标准是：如果一个约束算错了会导致方案不可行，或者会让报表口径失真，它就必须写在代码里。

04 AI 工作流 (1:30-2:15)

【04】 工作流用 LangGraph 编排，注册进图的是 18 个节点，由 19 条边语句展开成 25 条具体边，其中有 5 组条件分支。

【04】 主链路是：载入任务 → 数据感知 → 约束解析 → 规则校验 → 方案生成 → 任务排程 → 巡检处理 → 故障诊断 → 报告生成 → 人工确认 → 确认后建单下发。

【04】 这里有两个工程难点我想专门说。

【04】 第一个：人工确认没有用 LangGraph 的 `interrupt()`。

【04】 方案文档建议用 `interrupt()` 挂起图等待人工。但它依赖检查点与正在执行的图并发交互，在本机环境下会触发原生崩溃；而且进程内状态在多 worker 部署下无法共享。

【04】 所以我改成「两阶段执行 + 数据库持久化状态」：阶段一跑到人工确认节点就停住，把候选方案和状态写进 `ai_agent_task`，任务状态置为 `waiting_confirmation`；阶段二调用 `/confirm` 接口时，把确认结果注入初始状态重跑整图。

【04】 业务硬约束没有变 —— 仍然必须人工确认才能下发；换来的是可水平扩展、可审计。

【04】 第二个：约束松弛的重排循环必须设上限。

【04】 当所有候选方案都不满足约束时，「方案生成」和「约束松弛」两个节点会互相触发。早期版本没有上限，警告列表每轮翻倍，最终触发内存溢出。现在用 `relax_count` 计数，达到 2 次就转入异常处理终止，整图固定在 10 步以内收敛。

05 八类 Agent (2:15-2:50)

【05】 八类 Agent 我分三组讲。

【05】 第一组是业务执行类：智能工单调度（多方案比选 → 人工确认 → 建单下发，子任务按公式计算）、智能故障诊断（检索知识库加 LLM 增强，输出故障类别、根因与处置措施）、智能巡检报告（汇总巡检记录与异常项，生成结构化结论）。

【05】 第二组是分析与表达类：智能运维建议（基于历史工单与故障分布给出运维重点）、智能风控（识别超期、高频故障、低效巡检等风险信号）、智能报告（日/周/月/即时四类）、智能数据分析（对统计日表做趋势、对比与下钻）。

【05】 第三组只有一类：编排。它不直接产出业务结果，专门负责调度其它 Agent。

【05】 这一层的原则是：Agent 只负责「生成与解释」，可行性由硬约束和求解器保证 —— 换句话说，换一个更强的模型，也不会改变方案的可行性结论。

【05】 补充一下 LLM 的接入现状：provider 是 `deepseek`，model 是 `deepseek-chat`，健康检查返回 `status = ok`。没有配置 API Key 时会自动降级为规则引擎，功能不中断。

【05】 知识库检索用的是内置 TF-IDF + 余弦相似度，中文按 bigram 切分以提升召回 —— 这样不需要引入分词依赖和外部向量库就能开箱运行。

06 数据权限 (2:50-3:20)

【06】 数据权限分四个范围，由小到大：个人数据、站点数据、项目数据、平台数据。

【06】 个人数据是运维人员的默认范围，只能看自己的工单、巡检与故障记录；站点数据是本站点范围内全部记录；项目数据覆盖项目下所有站点；平台数据是全平台。

【06】 越权判断用一张顺序常量表（个人 0 → 站点 1 → 项目 2 → 平台 3）来做。

【06】 权限判定有两句话要说清：

【06】 有效权限 = 所有启用角色权限的并集，而且权限点自身也须启用；

【06】 用户没有任何角色时权限是空集，所有受保护接口返回 403。

【06】 这里还有三个容易做错的点：

【06】 第一，角色被停用 → 该角色贡献的权限整块失效，不是取最严；

【06】 第二，权限点被停用 → 即使角色仍引用它，也不生效；

【06】 第三，用户无任何角色 → 权限空集。

【06】 最后强调一句：前端的菜单裁剪和按钮隐藏只是体验优化 —— 最终防线是后端的 `require_permission()`。

07 验证结果 (3:20-3:55)

【07】 验证分三层，共 182 项，全部可重复执行。

【07】 **第一层，后端冒烟测试 96 项。** 它会自建独立的服务和数据库，与开发环境互不干扰。覆盖认证鉴权、工单闭环（含三类公式校验和状态机非法迁移拦截）、故障闭环（含高风险关键词自动判定危急）、附件上传（含非法类型拒绝和归属回填业务单据）。还包括**负向用例**——比如不存在的静态文件必须返回 404 而不是 500。

【07】 **第二层，真实服务联调 54 项。** 打的是真正跑着的 8010 和 5185 端口。这一层验证到的链路我念几条：接口数 **103**（从活服务取 openapi.json）；LangGraph workflow 挂起于「待人工确认」；**节点追踪落库 10 条**；多方案比选选中方案 A、得分 **69.09**；人工确认后**建单成功，8 个子任务、覆盖 8 个站点、通知 6 人**；故障诊断的 **LLM 增强 = True**；RAG 检索命中 3 条；报告生成的**六大内容模块齐全**；Excel 导出 116 行可下载。

【07】 **第三层，前端模块编译 32 项。** 用 Vite 实时转换每一个页面模块，确认不是「首页返回 200」就算过。

【07】 所谓可重复，是指**连续跑两遍结果一致**——这本身就证明了写操作的幂等性。

08 已知限制 (3:55-4:25)

【08】 这一页我主动交代限制，不用示例数据撑场面。

【08】 **第一，外部平台全部没有真实对接。** 储能运营平台、充电桩平台、电力市场、天气、地图路径规划——**都只有配置位和调用封装**，没有可用凭据，未做联调。未配置时相关章节**自动省略，不生成占位数字**。

【08】 **第二，小程序端与员工端没有建工程。** 后端接口已就绪（比如「我的子任务」接口），但原生小程序工程没有创建。本次范围确认的是「后端 + LangGraph Agent 中心 + Vue3 管理后台」。

【08】 **第三，知识库只有 6 条演示数据**——SOP 3 条、故障案例 2 条、设备手册 1 条。检索链路是完整可用的，但**语料规模确实不足以支撑生产级诊断质量**，需要导入真实运维资料后才能评估。

【08】 **第四，交付文档没有全部单独成文。** 架构、数据库与接口设计已写入 README；但**测试报告、用户手册、运维手册、AI 评估报告没有单独成文**——三套验证脚本本身可以作为测试证据。

【08】 **第五，一些技术性简化：**向量库用的是内置实现，不是 Milvus 或 Infinity；报告推送的微信、飞书、邮件通道只有代码路径，无凭据时安全跳过。

【08】 我的态度是：**宁可少写几个功能，也不放一个看起来能跑、实际经不起查的东西上去。**

09 系统架构 (4:25-4:50)

【09】 架构分四层。

【09】 **表现层**是 Vue 3.5 加 Vite 6 加 Ant Design Vue 加 ECharts，28 个 Vue 页面、23 个页面路由；菜单与按钮按服务端权限动态裁剪。

【09】 **接口层**是 FastAPI 加 Pydantic v2，JWT 用 python-jose，密码哈希用 bcrypt；**103 个接口路径、114 个操作**，统一走 `require_permission()` 鉴权，`apply_data_scope()` 做四范围过滤。

【09】 **服务层**是核心：求解器的硬约束与 CP-SAT、工单的子任务公式、RBAC 权限并集、规则版本与审计、报告生成、LangGraph 编排——全部集中在这一层，接口层不重复判断。

【09】 **数据层**是 SQLAlchemy 2.0 异步，**默认 SQLite**（用 aiosqlite，配 NullPool + WAL 模式），**可以切 PostgreSQL**——psycopg 驱动已经装好，改 `DATABASE_URL` 就能切，模型层没有使用任何方言。32 张表、117 个索引。

【09】 右上角这一栏是规则版本化：`rule_version` 表有 8 个自有字段，存硬约束、软约束和快照。激活新版本时会先把它行置为非激活，保证同一时刻只有一个生效版本，可以回滚。

【09】 所以结论是：规则不是写死在代码里的常量 —— 调整巡检频率、故障 SLA、软约束权重，都可以发新版本，出问题能回滚。

10 三个真正卡住我的问题（4:50-5:50）★ 全场重点

【10】 最后讲三个真正卡过我的问题。这段是我最想讲的。

问题一：一个无限循环把内存吃爆了

【10】 当所有候选方案都不满足约束时，「方案生成」和「约束松弛」两个节点会互相触发，`validation_warnings` 每轮翻倍：1 → 2 → 4 → → 131072，最终触发内存溢出，把机器拖慢。

【10】 排查过程中我一度怀疑是 OR-Tools、langgraph 版本、SQLite 锁或者并发问题 —— 但我单独跑 CP-SAT 求解只要 0.018 秒，全都不是原因。

【10】 真正的根因是缺一个重试计数器。加上 `relax_count` 并设置上限 2 之后，整图固定在 10 步以内收敛，不再出现内存溢出。

【10】 这件事让我记住：排查问题时，先确认自己怀疑的方向有没有被证据支持。||

问题二：方案文档建议的 `interrupt()` 在本机跑不通

【10】 方案文档建议用 LangGraph 的 `interrupt()` 来做人工确认挂起。但它在我的环境下会触发原生崩溃 —— 原因是它依赖检查点与正在执行的图并发交互；而且进程内状态在多 worker 部署下无法共享。

【10】 我改成了两阶段执行加数据库持久化状态：阶段一跑到确认节点就停，状态落库；阶段二用 `/confirm` 把确认结果注入初始状态重跑整图。

【10】 业务硬约束没有打折 —— 仍然是「必须人工确认才能下发」；换来的是可水平扩展、可审计，确认记录落在 `ai_agent_task.confirmation` 里。

【10】 我也要如实说一句：代码里还留有几处「`interrupt`」的旧注释没有清理，这是遗留的技术债。||

问题三：自己发现的 404 变 500

【10】 这个是我在核对功能缺口时发现的。访问一个不存在的静态图片，服务端返回了 500，而不是 404。

【10】 根因是兜底的 404 处理器无差别返回 `JSONResponse`，把 `StaticFiles` 抛出的 `HTTPException` 当成了服务器内部错误。关键点是：在异常处理器里再次 `raise`，会逃逸到兜底的 500 处理器。

【10】 正确做法是按 `status_code` 原样返回响应：API 路径给统一 JSON 结构，静态资源给纯文本。

【10】 修完之后我补了 2 条回归断言 —— 不存在的静态文件必须返回 404，不存在的 API 返回统一 JSON。||

【10】 这三件事合起来说明一个态度：方案文档给的是方向，真正跑起来才会暴露边界；文档建议的做法也不一定可用，要能说清为什么换。

11 关键取舍与预设问答（5:50-6:50）

【11】 这一页我把可能被问到的问题和答案列出来。每个偏离方案的决定，都有可核对的理由。

【11】 问：为什么没用 LangGraph 的 `interrupt()`？

【11】 答：它依赖检查点与执行中的图并发交互，**在本机会触发原生崩溃**，且进程内状态多 worker 无法共享。改用两阶段加数据库持久化状态——依据写在 `nodes.py:1442-1455`。

【11】 问：LLM 到底用在哪？会不会算错？

【11】 答：只做方案解释、报告生成与语言表达。**硬约束、子任务公式、可行性判定全部在代码里**，模型算错也不影响方案。

【11】 问：知识库只有 6 条，RAG 有意义吗？

【11】 答：**检索链路是完整可用的**（含中文 bigram 提升召回），但语料规模确实不足以支撑生产级诊断质量。需要导入真实资料后再评估——我不会拿 6 条数据说 RAG 效果好。

【11】 问：为什么默认 SQLite，方案写的是 PostgreSQL？

【11】 答：SQLite 是为了开箱即跑、降低演示门槛。psycopg 驱动已安装，切换只需改 `DATABASE_URL`，模型层未使用任何方言。

【11】 问：重排会不会死循环？

【11】 答：**不会**。重排有次数上限（2 次），达到上限转异常处理终止，整图固定 10 步内收敛；**已完成子任务锁定不参与重排**。

【11】 原则是：**偏离方案可以，但每一个偏离都要能说清原因，并且能当场核对**。

12 总结 (6:50–7:10)

【12】 总结一下。

【12】 这个平台把运维规则搬进了代码，三个特点：

【12】 **可配置**——巡检频率、故障 SLA、软约束权重都在规则中心，可发版、可回滚；

【12】 **可解释**——每套方案带评分与推荐理由，LLM 只做表达；

【12】 **可追溯**——确认记录、节点追踪、审计日志全部落库。

【12】 目前 **182 项验证全部通过**，**103 个接口**在真实服务上可用。

【12】 如果继续做，下一步是三件事：**导入真实运维知识**（现在只有 6 条）、**接通一个外部平台**（优先充电桩平台或储能平台）、**清理过期表述并补齐文档**（测试报告、用户手册、运维手册需要单独成文）。

【12】 感谢各位老师，请提问。

三、现场演示动线 (7:10 起，约 4–5 分钟)

只走一条主线，**不要乱翻页**。提前把浏览器开好、`admin` 登录好。

步	操作	要念的一句话
1	首页看板	「这就是四条主线的总览：工单、巡检、故障、报告。」
2	参数与规则 → 规则版本	「当前生效版本 v1.0.0，里面有巡检频率 日 30 / 周 4 / 月 1。」
3	AI Agent 中心	「八类 Agent，右边是实时任务数。」
4	智能工单调度 → 新建任务 → 开始	「 workflow 跑起来了 —— 多方案比选，每套带评分。」
5	停在「待人工确认」	「看，它停在这里了，没有自动下发。这就是人工确认。」
6	选方案 A → 确认	「确认后建单：8 个子任务、覆盖 8 个站点、通知 6 人。」
7	工单详情 → 巡检录入 → 传照片	「照片走真实上传，落盘后自动绑定到这张单据。」
8	智能故障诊断	「输入现象，它检索知识库加 LLM，给出根因和措施。」
9	运维报告 → 生成	「六大模块齐全：运行概览、设备状态、根因分析、运维建议.....可以追问下钻。」

- 【12】 如果时间紧，只演第 5 步和第 6 步 —— 「停住不自动下发」这一下最能说明设计意图。
- 【12】 第 7 步的照片上传是第二优先，能证明附件真的落盘了。

四、演示前 5 分钟检查清单

- ☐ 后端 `python run.py` 已起 (8010) , 前端 `pnpm run dev` 已起 (5185)
- ☐ 浏览器已登录 `admin / admin123`
- ☐ 确认 AI 健康检查是 ok (`/api/v1/ai/health`) —— 否则诊断会降级
- ☐ 语速：第一段刻意放慢
- ☐ 数字念准：29392 行、17502/11890、32 张表、103 接口、8 类 Agent、18 节点、182 项、96/54/32
- ☐ 讲第 10 页三个问题时不要道歉、不要心虚 —— 这是加分项
- ☐ 两句金句说完停一秒：
「硬约束代码化，LLM 只负责解释与表达。」
「宁可少写几个功能，也不放一个看起来能跑、实际经不起查的东西上去。」
- ☐ 超时优先砍第 09 页，第 03、07、08、10 页不能砍

五、预设问答（10 分钟问答用）

按「先给结论，再给依据」回答。每题控制在 40 秒内。

可能的提问	回答要点
为什么没用 LangGraph 的 <code>interrupt()</code> ?	它依赖检查点与执行中的图并发交互， 在本机会触发原生崩溃 ，且进程内状态多 worker 无法共享。改用两阶段 + 数据库持久化状态；业务硬约束不变。依据 <code>nodes.py:1442-1455</code> 。
LLM 会不会把方案算错？	不会影响可行性。 硬约束、子任务公式、可行性判定全在代码里 （ <code>validate_hard_constraints</code> 、 <code>calc_subtask_count</code> ），LLM 只做解释与表达。
子任务公式为什么要硬编码？	它 直接决定派工量、工时估算和考核口径 ，算错一个整月报表失真。而且公式只有一份实现，求解器与接口共用，不重复写。
知识库只有 6 条，RAG 有意义吗？	检索链路完整可用（TF-IDF + 余弦 + 中文 bigram），但 语料规模不足以支撑生产级诊断质量 ，需导入真实资料后再评估。
为什么默认 SQLite？	开箱即跑、降低演示门槛。 psycopg 已装，改 <code>DATABASE_URL</code> 即切 PostgreSQL ，模型层无方言。
CPSAT_ENABLED 关掉会怎样？	退化为启发式求解， 方案仍然可用 ，只是不再做精确优化；关闭时会在方案说明里写明原因。超时默认 5 秒，并行 4 worker。
重排会不会死循环？	不会。 <code>MAX_RELAX_RETRY = 2</code> ，达上限转 <code>exception_handler</code> 终止，整图 10 步内收敛；已完成子任务锁定不重排。
权限模型有什么坑？	三个：① 角色停用 → 该角色贡献的权限 整块失效 （不是取最严）；② 权限点停用 → 角色引用也不生效；③ 无角色 → 权限空集，403。
审计日志能篡改吗？	接口层只有查询，写入路径唯一（ <code>services/audit.py</code> ），规则版本激活时会先把其它行置为非激活，保证同一时刻只有一个生效版本。
182 项验证怎么保证可重复？	冒烟测试自建独立服务与数据库；连续跑两遍结果一致，说明写操作是幂等的。
为什么有 404 变 500 这种问题？	兜底 404 处理器无差别返回 <code>JSONResponse</code> ，把 <code>StaticFiles</code> 的 <code>HTTPException</code> 当成内部错误；在异常处理器里再 <code>raise</code> 会逃逸到 500 处理器。已按 <code>status_code</code> 原样返回，并补 2 条回归断言。
最难的地方是什么？	把「人的经验」翻译成可执行的形式化约束 ：比如日 30 / 周 4 / 月 1 这种频率映射，得跟业务逐条核对过才敢写进代码。

六、被问到不会的问题怎么办

【12】 「这个问题我目前没有验证过，不能凭印象回答。我记下来，答辩结束后查清再回复您。」

不要现场猜。 这个项目的立场就是「编不出来的数据我不编」——现场编一个答案，会把前面 182 项可重复验证的可信度一起赔进去。

七、数据依据索引（被追问时翻这里）

数字	依据
17502 行 / 51 文件	<code>backend/app</code> 下 <code>.py</code> 统计 (字节级计数)
11890 行 / 33 文件	<code>frontend/src</code> 下 <code>.vue</code> 28 个 11099 行、 <code>.js</code> 4 个 542 行、 <code>.css</code> 1 个 249 行
32 张表 / 117 索引	<code>__tablename__</code> 命中 32 处; <code>sqlite_master</code> 查询
103 接口 / 114 操作	<code>GET /openapi.json</code> (GET 62 / POST 43 / PUT 7 / DELETE 2)
8 类 Agent	<code>core/enums.py:111-119</code> <code>AgentType</code> ; <code>GET /api/v1/ai/agent/center</code>
18 节点 / 25 边 / 5 条件边	<code>graph.py</code> 的 <code>add_node</code> 18 处、14 条 <code>add_edge</code> + 5 条 <code>add_conditional_edges</code>
23 个顶层函数	<code>nodes.py</code> : 17 个节点函数 + 6 个辅助函数; <code>NODE_LABELS</code> 17 项
日 30 / 周 4 / 月 1	<code>services/work_order.py:39-43</code> <code>FREQUENCY_PER_CYCLE</code>
消缺 = 1、特巡 = 站点 × 次数	<code>work_order.py:68-90</code> <code>calc_subtask_count()</code>
<code>MAX_RELAX_RETRY = 2</code>	<code>graph.py:43</code> ; 判断逻辑 <code>graph.py:46-52</code>
数据权限 4 范围 / 序 0-1-2-3	<code>core/enums.py:94-107</code>
LLM deepseek / status ok	<code>GET /api/v1/ai/health</code>
知识库 6 条	<code>GET /api/v1/ai/knowledge/categories</code> (SOP 3 / 故障案例 2 / 设备手册 1)
CPSAT true / 5.0s / 4 workers	<code>config.py:72-73</code> ; <code>solver.py:279-280</code>
<code>rule_version / v1.0.0 active</code>	<code>models/system.py:154-166</code> ; <code>bootstrap.py:249-266</code>
182 项	冒烟 96 (<code>smoke_report.txt</code>) + 联调 54 + 前端 32

【12】 两个引用提醒

【12】 ① `GET /api/v1/ai/agent/center` 的 `total_tasks`、`report_count` 是活数据 (实测 1 分钟内从 5/10 变成 6/11)，答辩时若要点这些数字，先现取一次。

【12】 ② 代码中 `graph.py:202-203`、`nodes.py:13`、`agent_service.py:178`、`models/ai.py:34` 仍写「interrupt 挂起/恢复」，`graph_info()` 还硬编码 `"checkpointner": "InMemorySaver"`。这些是过期注释，答辩时按「两阶段执行 + 数据库持久化」表述。